

Software Requirements Specification (SRS) — File Compression Tool (Basic ZIP Implementation)

Project: File Compression Tool (Basic ZIP Implementation)

Version: 1.0

Authors: Adarsh Prakash (PES2UG24CS027), Aakash Agarwal (PES2UG24CS009), Abhijna Marathe (PES2UG24CS020), Abhirup Roy (PES2UG24CS025)

Date: 05-09-2026

Status: Draft — Submitted for Review

Revision history

Version	Date	Author	Change summary	Approval
1.0	05-09-2026	Project Team	SRS for File Compression Tool with diagrams embedded	Pending

Approvals

Role	Name	Signature / Email	Date
Course Coordinator	< Name >	< Signature / Email >	< Date >

Table of Contents

1. Introduction
2. Overall description
3. External interfaces
4. System features (detailed)
5. Non-functional requirements (detailed)
6. Quality attributes & Acceptance tests
7. UML Use-Case Diagram
8. Requirements Traceability Matrix (RTM)

1. Introduction

1.1 Purpose

This document is a Software Requirements Specification (SRS) for a File Compression Tool implementing a basic ZIP-compatible archive workflow. It defines the functional and non-functional requirements, external interfaces, security requirements, verification criteria, and traceability needed to develop and assess the project.

1.2 Scope

Covers selecting input files and folders, compressing them into a single archive, extracting archive contents, viewing archive information, validating archive paths, handling invalid/corrupted input, and reporting operation status. The project is a basic ZIP implementation and does not require advanced enterprise archive management, cloud storage, or distributed processing.

1.3 Audience

Developers, QA Engineers, Student Project Teams, Instructors, and Assessment Evaluators.

1.4 Definitions

List of acronyms: SRS, ZIP, CRC, UI, CLI, API, I/O, UTF-8, DOS, NFR, FR, RTM.

2. Overall description

2.1 Product perspective

The File Compression Tool is a standalone software application that reads files and directories from local storage, compresses their contents into a ZIP archive, and extracts ZIP archives to a user-selected destination. It contains file-selection and I/O handling, compression and archive-management logic, ZIP record generation/parsing, integrity validation, and a user interface.

2.2 Major product functions (detailed)

- Select files and folders for compression
 - Create a ZIP archive from selected inputs
 - Preserve file names and directory structure
 - Extract archive entries
 - List archive contents and metadata
 - Validate archive structure and detect invalid/corrupted data
 - Show operation progress, success, and error messages
 - Prevent unsafe extraction paths
 - Handle duplicate names and existing destination files
 - Clean up temporary resources after operations

2.3 User roles and characteristics (expanded)

User: basic computer/file-system knowledge; expects clear controls and understandable errors.

Developer/Maintainer: needs modular compression/archive code, diagnostics,

and logs.

Assessment Evaluator: checks functional requirements, security controls, performance targets, and test evidence.

2.4 Operating environment

Desktop environment such as Windows, Linux, or macOS with access to local file storage.

The application may be implemented as a command-line or desktop GUI program. It requires sufficient RAM and disk space for input files, temporary data, and generated archives.

2.5 Constraints

The implementation shall use a ZIP archive structure and shall not assume that all files fit into memory at once. File paths must be handled safely across supported operating systems. The basic implementation may limit advanced ZIP features such as encryption, multi-volume archives, or uncommon compression methods unless explicitly implemented.

3. External interface requirements

3.1 User interfaces

Primary UI: command-line prompts or a simple desktop GUI with clear actions for Compress, Extract, List/Inspect, destination selection, and status/error reporting. Accessibility: readable text, clear error messages, keyboard-operable controls where a GUI is used, and no reliance on color alone to communicate status.

3.2 Hardware interfaces

Not applicable — this is a desktop/CLI software tool with no dedicated hardware peripherals. Interaction with local storage occurs through standard operating-system file-system interfaces, listed under Software interfaces below.

3.3 Software interfaces

- Local file system interface for reading input files and writing archives
 - Local file system interface for creating extraction directories and output files
 - Optional progress/status console or GUI controls
 - Standard operating-system file/path services
- ZIP archive parser and writer module
 - Compression/decompression library or project compression implementation
 - File-system API for file metadata and I/O
 - Optional logging module for audit/diagnostic events

3.4 Communications

- All operations are local unless explicitly extended with another interface.
 - File reads/writes shall report I/O failures.
 - Long-running operations shall provide progress or status information where practical.
 - The tool shall not transmit file contents over a network as part of normal operation.

4. System features (detailed)

Each requirement below includes acceptance criteria and a reference test case. IDs follow FCT-F-####.

4.1 File Selection and Compression

Description: Requirements FCT-F-001 to FCT-F-008 cover input selection, ZIP archive creation, directory preservation, supported compression, progress/status reporting, extraction destination selection, entry extraction, and path-traversal protection.

Req ID	Requirement	Type	Priority	Source/Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
FCT-F-001	The system shall allow the user to select one or more files and folders as compression inputs.	Functional	High	User	AC-FCT-F-001: Selected inputs are accepted and displayed before compression. Test: TC-CMP-01	Local file-system access
FCT-F-002	The system shall create a ZIP archive at a user-selected output location.	Functional	High	User	AC-FCT-F-002: A readable ZIP archive is created at the requested path. Test: TC-CMP-02	Write permission required
FCT-F-003	The system shall preserve the relative directory structure of selected folders inside the archive.	Functional	High	User / Business	AC-FCT-F-003: Extraction recreates the expected hierarchy. Test: TC-CMP-03	Safe path normalization
FCT-F-004	The system shall compress file data using the supported ZIP compression method.	Functional	High	Developer	AC-FCT-F-004: Compressed entries can be decompressed successfully. Test: TC-CMP-04	Compression module

FCT-F-005	The system shall report progress or status while processing a multi-file or long-running compression operation.	Functional	Medium	User	AC-FCT-F-005: User receives visible processing status. Test: TC-UX-01	UI/CLI dependent
FCT-F-006	The system shall allow the user to select a ZIP archive and an extraction destination.	Functional	High	User	AC-FCT-F-006: Valid archive and destination are accepted. Test: TC-EXT-01	Read/write permission required
FCT-F-007	The system shall extract supported entries while recreating the stored directory structure.	Functional	High	User	AC-FCT-F-007: Extracted files match the expected contents and structure. Test: TC-EXT-02	Archive parser
FCT-F-008	The system shall detect and reject unsafe extraction paths that escape the selected destination directory.	Functional	High	Security	AC-FCT-F-008: Path traversal entries are rejected or safely normalized. Test: TC-SEC-01	Path traversal protection

4.2 Archive Extraction

Description: Requirements FCT-F-009 to FCT-F-015 cover archive validation, listing, duplicate-file handling, entry integrity verification, binary-safe preservation, temporary-resource cleanup, and operation summary reporting.

Req ID	Requirement (shall...)	Type	Priority	Source/Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
FCT-F-009	The system shall validate ZIP structure and reject malformed or unsupported archives with an understandable error.	Functional	High	Security / Reliability	AC-FCT-F-009: Invalid archive is rejected without silent data loss. Test: TC-ERR-01	Archive validation
FCT-F-010	The system shall provide	Functional	Medium	User	AC-FCT-F-010: Listing	Archive parser

	a way to list archive entries before extraction.				shows all readable entries without extracting them. Test: TC-LST-01	
FCT-F-011	The system shall handle duplicate output file names according to an explicit overwrite, skip, or rename policy.	Functional	High	User	AC-FCT-F-011: Existing-file behavior follows the selected policy. Test: TC-EXT-03	Overwrite policy
FCT-F-012	The system shall verify supported entry integrity using available ZIP integrity information and report mismatches.	Functional	High	Security / Reliability	AC-FCT-F-012: Corrupted entry is detected. Test: TC-SEC-02	CRC/integrity validation
FCT-F-013	The system shall preserve binary file contents without text conversion during compression and extraction.	Functional	High	User / Developer	AC-FCT-F-013: Binary files are byte-for-byte identical after round trip. Test: TC-INT-01	Binary-safe I/O
FCT-F-014	The system shall clean up temporary resources created during compression or extraction	Functional	Medium	Operations	AC-FCT-F-014: Temporary resources are removed after operation completion/f	Cleanup logic

	after success or failure.				ailure. Test: TC-OPS-01	
FCT-F-015	The system shall provide a final operation summary indicating success, failure, affected files, and output archive/path where applicable.	Functional	Medium	User	AC-FCT-F-015: Summary is displayed after each operation. Test: TC-UX-02	UI/CLI dependent

4.3 Archive Validation and Integrity

Requirements: FCT-F-009 and FCT-F-012 cover ZIP structure validation and entry integrity checking.

4.4 Archive Inspection and File Handling

Requirements: FCT-F-010, FCT-F-011, and FCT-F-013 cover archive inspection, duplicate-file handling, and binary-safe file handling.

4.5 Operation Control and Error Handling

Requirements: FCT-F-014 and FCT-F-015 cover temporary-resource cleanup and final operation reporting. Progress/status reporting is defined under Section 4.1.

5. Non-functional requirements (detailed)

NFRs below are measurable and tied to test plans. IDs FCT-NF-###.

Req ID	Requirement	Category	Priority	Acceptance criteria / Measurement
FCT-NF-001	For the reference test machine and normal local storage, compression and extraction shall achieve a 95th-percentile elapsed time of <= 2 seconds per 100 MB for the defined benchmark workload.	Performance	High	Benchmark results meet the target. Test: TC-PERF-01
FCT-NF-002	The application shall use no more than 512 MB peak working memory for a 1 GB sequential test workload under	Resource Efficiency	Medium	Measured peak application memory <= 512 MB. Test: TC-PERF-02

	reference conditions.			
FCT-NF-003	Error messages for invalid archives, permission failures, and I/O failures shall identify the failed operation and affected file or path where available.	Usability/Reliability	High	Each defined scenario contains operation + affected path information where available. Test: TC-UX-03
FCT-NF-004	For supported file types, a successful compress-extract round trip shall preserve file size and SHA-256 content hash.	Reliability/Data Integrity	High	Original and extracted files have identical size and SHA-256 hash. Test: TC-INT-02
FCT-NF-005	On each operating system included in the supported project scope, the tool shall correctly normalize and process native file-path separators and relative paths.	Portability	Medium	Path test suite passes on each supported OS. Test: TC-PORT-01

5.1. Security

5.1.1 Security Objectives

1. Protect local file and archive integrity by preventing unsafe paths, unintended overwrites, and undetected corruption during supported operations.
2. Minimize unintended disclosure by keeping file contents local and preventing sensitive payloads from being written to normal diagnostic logs.

5.1.2 Security Requirements

Req ID	Requirement (shall...)	Type	Priority	Acceptance criteria / Test case ref
FCT-SR-001	The system shall prevent archive path traversal by ensuring every resolved extraction path remains within the selected destination directory.	Security	High	TC-SEC-01
FCT-SR-002	The system shall validate ZIP structure and reject malformed archives before unsafe or unsupported entries are processed.	Security	High	TC-SEC-03
FCT-SR-003	The system shall verify supported entry integrity and report any detected integrity mismatch.	Security	High	TC-SEC-02

FCT-SR-004	The system shall not write file contents or sensitive payload data to normal diagnostic logs; logs shall contain only necessary operational metadata.	Security	High	TC-SEC-04
FCT-SR-005	The system shall require an explicit overwrite policy for existing destination files and shall default to non-destructive behavior unless overwrite is intentionally selected.	Security	High	TC-SEC-05

6. Quality attributes & Acceptance tests

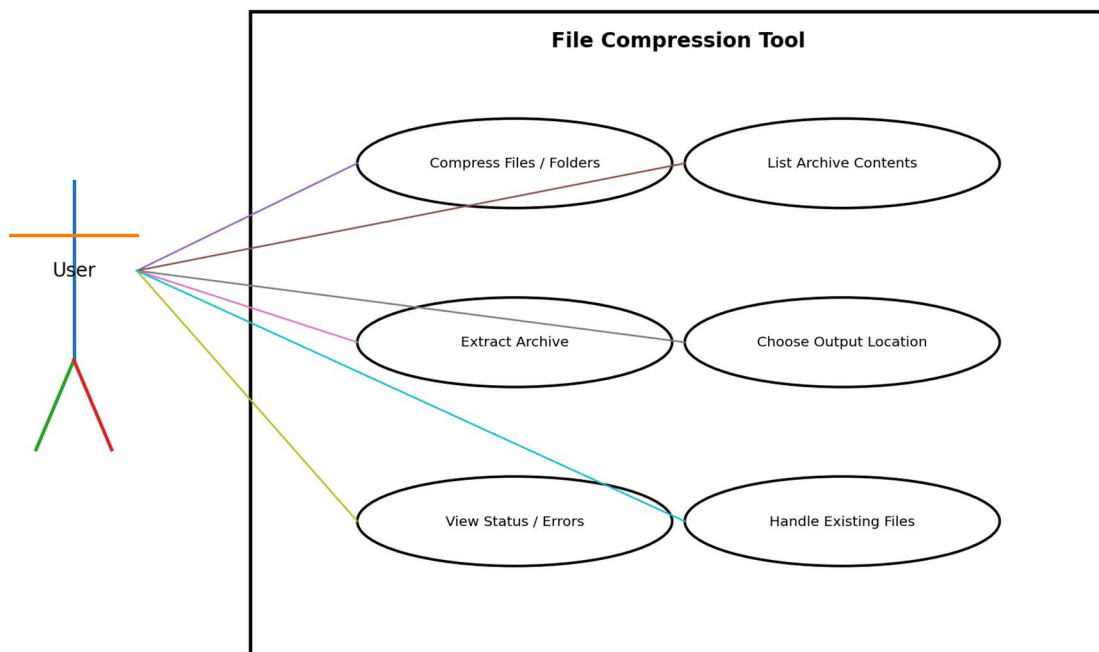
- Exit criteria for acceptance: All high-priority functional requirements implemented and verified, no critical NFR or security failures, and the RTM shows all required test cases mapped.
- Acceptance test suites: Compression, Extraction, Archive Inspection, Error Handling, Performance, Security, and Usability tests.

7. System models and diagrams

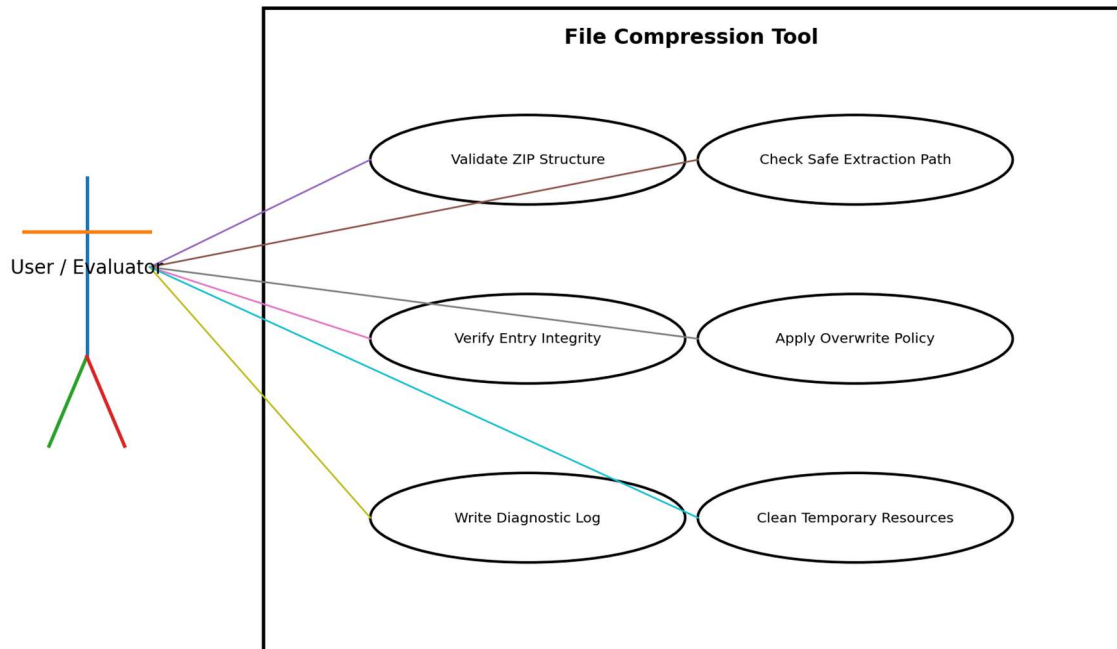
7.1 UML Use-Case diagram

Two UML use-case diagrams are embedded below: the primary user workflow and the archive/security-management workflow.

UML Use-Case Diagram 1 — Primary User Workflow



UML Use-Case Diagram 2 — Archive & Security Management



8. Requirements Traceability Matrix (RTM)

Req ID	Requirement short	Section ref / Design Spec	Module	Test case(s)	Status (N/P/A)	Comments
FCT-F-001	Select compression inputs	4.1 / DS-Input-01	Input Module	TC-CMP-01	N	
FCT-F-002	Create ZIP archive	4.1 / DS-Archive-01	Archive Writer	TC-CMP-02	N	
FCT-F-003	Preserve directory structure	4.1 / DS-Path-01	Path Module	TC-CMP-03	N	
FCT-F-004	Compress file data	4.1 / DS-Compress-01	Compression Module	TC-CMP-04	N	
FCT-F-005	Report compression status	4.1 / DS-UI-01	UI / CLI Module	TC-UX-01	N	
FCT-F-006	Select extraction destination	4.2 / DS-Extract-01	Extraction Module	TC-EXT-01	N	
FCT-F-007	Extract archive entries	4.2 / DS-Extract-02	Extraction Module	TC-EXT-02	N	

FCT-F-008	Block path traversal	4.2 / DS-Sec-01	Security Module	TC-SEC-01	N	
FCT-F-009	Reject malformed archive	4.3 / DS-Parser-01	Archive Parser	TC-ERR-01	N	
FCT-F-010	List archive contents	4.4 / DS-List-01	Archive Inspector	TC-LST-01	N	
FCT-F-011	Apply duplicate-file policy	4.4 / DS-Overwrite-01	Extraction Module	TC-EXT-03	N	
FCT-F-012	Verify entry integrity	4.3 / DS-CRC-01	Integrity Module	TC-SEC-02	N	
FCT-F-013	Preserve binary contents	4.4 / DS-IO-01	File I/O Module	TC-INT-01	N	
FCT-F-014	Clean temporary resources	4.5 / DS-Cleanup-01	Resource Manager	TC-OPS-01	N	
FCT-F-015	Operation completion summary	4.5 / DS-UI-02	UI / CLI Module	TC-UX-02	N	
FCT-NF-001	Compression /extraction performance	5 / DS-Perf-01	Compression / Extraction	TC-PERF-01	N	
FCT-NF-002	Peak memory usage	5 / DS-Perf-02	Application	TC-PERF-02	N	
FCT-NF-003	Actionable error messages	5 / DS-UX-01	UI / CLI Module	TC-UX-03	N	
FCT-NF-004	Round-trip data integrity	5 / DS-Int-01	Compression / Extraction	TC-INT-02	N	
FCT-NF-005	Portable path handling	5 / DS-Port-01	File / Path Module	TC-PORT-01	N	
FCT-SR-001	Path traversal protection	5.1.2 / DS-Sec-01	Security Module	TC-SEC-01	N	
FCT-SR-002	ZIP structure validation	5.1.2 / DS-Sec-02	Archive Parser	TC-SEC-03	N	
FCT-SR-003	Entry integrity verification	5.1.2 / DS-Sec-03	Integrity Module	TC-SEC-02	N	
FCT-SR-004	Sensitive-data-safe logging	5.1.2 / DS-Log-01	Logging Module	TC-SEC-04	N	
FCT-SR-005	Non-destructive overwrite policy	5.1.2 / DS-Sec-04	Extraction Module	TC-SEC-05	N	